SEMT20003 Practice Paper 1

Foundations, Regression, and Basic Architectures

Time: 3 hours **Total Marks:** 100

Part I: Multiple Choice Questions (25 marks)

Answer all questions. Each question has 4 possible options. Between ONE and THREE of these options may be correct. For full marks (5 marks per question), you must state exactly the correct options. No partial credit is awarded.

1. **Given matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and matrix $\mathbf{B} \in \mathbb{R}^{n \times p}$, which of the following operations are mathematically valid?**

   A) $\mathbf{AB}$

   B) $\mathbf{BA}$

   C) $\mathbf{A}^T\mathbf{A}$

   D) $\mathbf{A} + \mathbf{B}$

2. **Which of the following statements regarding regression families (linear vs. quadratic) are true?**

   A) Quadratic regression models are non-linear with respect to their input features.

   B) Quadratic regression models are non-linear with respect to their weight parameters.

   C) A linear regression model can perfectly fit a dataset generated by a strictly quadratic function.

   D) Both linear and quadratic regression models can be optimized using Gradient Descent.

3. **Regarding the Mean Squared Error (MSE) loss function for regression:**

   A) It is calculated by taking the sum of the absolute differences between predictions and targets.

   B) It penalizes large prediction errors more heavily than small prediction errors.

   C) It is formulated as $L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$.

   D) It is robust to outliers compared to Mean Absolute Error.

4. **Which of the following are properties of the ReLU (Rectified Linear Unit) activation function?**

A) It is mathematically defined as $f(x) = \max(0, x)$.

B) Its derivative is exactly 1 for all $x > 0$.

C) Its output is bounded between 0 and 1.

D) Its derivative is exactly 0 for all $x < 0$.

5. **Which of the following statements correctly describe standard Gradient Descent?**

   A) The model weights are updated in the direction of the gradient of the loss function.

   B) The step size of the weight update is controlled by a hyperparameter called the learning rate.

   C) A single update step utilizes the entire training dataset to compute the gradient.

   D) It is mathematically guaranteed to find the global minimum for any neural network architecture.

Part II: Programming-related Questions (35 marks)

Question 1: Multivariate Linear Regression Model (15 marks)
You are given a dataset with N samples and D features. Write a Python function using NumPy that implements the forward pass of a multivariate linear regression model and calculates the Mean Squared Error (MSE) loss.

Your function `linear_regression_forward(X, y, W, b)` should accept:

- `X`: A NumPy array of shape `(N, D)` representing the input features.
- `y`: A NumPy array of shape `(N,)` representing the true target values.
- `W`: A NumPy array of shape `(D,)` representing the weights.
- `b`: A float representing the bias.

The function should return a tuple `(predictions, loss)`.

Question 2: Gradient Descent Training Loop (20 marks)
Using the context from Question 1, write a complete training loop to optimize the weights `W` and bias `b` using basic Gradient Descent. Assume you already have access to the arrays `X` and `y`. Initialize `W` with zeros and `b` as zero. Run the loop for `epochs = 100` with a `learning_rate = 0.01`. Inside the loop, you must explicitly compute the gradients of the MSE loss with respect to `W` and `b`, update the parameters, and store the loss at each epoch in a list called `loss_history`.

Part III: Analytic Questions (40 marks)

Question 1: Gradient Derivation (20 marks)
Given a multivariate linear regression model, the prediction $\hat{y}_i$ for a single data point $\mathbf{x}_i \in \mathbb{R}^D$

is defined as:

$$\hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

The Mean Squared Error loss over N data points is defined as:

$$L = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Analytically derive the gradient of the loss with respect to the weight vector $\mathbf{w}$ (i.e., $\frac{\partial L}{\partial \mathbf{w}}$) and the gradient with respect to the bias b (i.e., $\frac{\partial L}{\partial b}$). Show all steps clearly.

Question 2: 2-Layer Neural Network Forward Pass (20 marks)
Consider a simple 2-layer Neural Network designed for a regression task. The network receives an input vector $\mathbf{x} \in \mathbb{R}^{d_{in}}$, has a single hidden layer with d_h neurons utilizing a ReLU activation function, and outputs a scalar prediction $\hat{y} \in \mathbb{R}$.

- **a) (10 marks):** Mathematically define the sequence of operations for the forward pass of this network. Clearly state the dimensions of all weight matrices and bias vectors introduced.
- **b) (10 marks):** Discuss analytically why non-linear activation functions (like ReLU) are strictly necessary in the hidden layers of a deep neural network. Use mathematical reasoning to show what happens if the network only uses linear activations.

Mark Scheme: Practice Paper 1

Part I: Multiple Choice Questions (25 marks)

5 marks per question. Must state exactly the correct combinations. Partial credit is 0.

1. **A, C**
   Rationale: A is valid ($m \times n$ multiplied by $n \times p$ results in $m \times p$). C is valid ($n \times m$ multiplied by $m \times n$ results in $n \times n$). B is invalid (inner dimensions p and m do not match). D is invalid (dimensions must be identical for addition).

2. **A, D**
   Rationale: Quadratic models include features squared (non-linear to inputs), making A true. They remain linear with respect to their weights (e.g., $w_1x^2 + w_2x$), making B false. A linear model cannot perfectly fit a quadratic curve, making C false. Both can be optimized via Gradient Descent, making D true.

3. **B, C**
   Rationale: MSE squares the errors, meaning large errors are penalized exponentially more than small ones (B is true). C is the exact formula for MSE. A describes Mean Absolute Error. D is false; squaring errors makes MSE highly sensitive to outliers.

4. **A, B, D**
   Rationale: A is the exact definition of ReLU. B is true because the slope of $y = x$ is 1. D is true because the slope of $y = 0$ is 0. C is false because ReLU outputs can go up to positive infinity.

5. **B, C**
   Rationale: B is true (learning rate scales the update step). C is true (Standard/Batch Gradient Descent uses the whole dataset; Mini-batch or Stochastic do not). A is false (weights are updated in the negative direction of the gradient, $-\nabla L$). D is false (it only guarantees finding a global minimum for convex loss landscapes; NNs are highly non-convex).

Part II: Programming-related Questions (35 marks)

Question 1 (15 marks)

- **5 marks:** Correctly calculating the predictions via dot product.
- **5 marks:** Correctly calculating the squared errors.
- **5 marks:** Correctly calculating the mean of the squared errors to return the final MSE.

```python
import numpy as np

def linear_regression_forward(X, y, W, b):
    # Calculate predictions (5 marks)
    predictions = np.dot(X, W) + b

    # Calculate Mean Squared Error (10 marks total)
    errors = predictions - y
    loss = np.mean(errors ** 2)

    return predictions, loss
```

Question 2 (20 marks)

- **3 marks:** Correctly initializing weights, bias, and loop.

- **5 marks:** Forward pass to get predictions.
- **8 marks:** Correctly calculating the gradients mathematically in code (`dW` and `db`).
- **4 marks:** Correctly updating weights using the learning rate and storing loss.

```python
import numpy as np

# Assume X, y are provided
N, D = X.shape
W = np.zeros(D)  # Init W
b = 0.0          # Init b
learning_rate = 0.01
epochs = 100
loss_history = []

for epoch in range(epochs):
    # Forward pass
    predictions = np.dot(X, W) + b
    loss = np.mean((predictions - y) ** 2)
    loss_history.append(loss)

    # Calculate gradients
    error = predictions - y
    # Gradient w.r.t W: (2/N) * X^T * error
    dW = (2 / N) * np.dot(X.T, error)
    # Gradient w.r.t b: (2/N) * sum(error)
    db = (2 / N) * np.sum(error)

    # Update parameters
    W = W - learning_rate * dW
    b = b - learning_rate * db
```

Part III: Analytic Questions (40 marks)

Question 1: Gradient Derivation (20 marks)

- **5 marks:** Applying the chain rule to the summation.
- **5 marks:** Correct derivative of the outer square function: $2(y_i - \hat{y}_i)$.
- **5 marks:** Correct inner derivative w.r.t $\mathbf{w}$ $(-\mathbf{x}_i)$ resulting in $\frac{\partial L}{\partial \mathbf{w}} = \frac{-2}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)\mathbf{x}_i$.
- **5 marks:** Correct inner derivative w.r.t b (-1) resulting in $\frac{\partial L}{\partial b} = \frac{-2}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)$.

(Note: Dropping the 2 or moving the negative sign around is acceptable as long as the algebra holds).

Question 2: 2-Layer Neural Network Forward Pass (20 marks)

Part a) (10 marks)

- **4 marks:** Defining hidden layer pre-activation $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$ and activation $\mathbf{h} = \text{ReLU}(\mathbf{z}_1)$.
- **2 marks:** Defining output layer $\hat{y} = \mathbf{W}_2\mathbf{h} + b_2$.

- **4 marks:** Correctly stating dimensions:

$$\begin{aligned}\mathbf{W}_1 &\in \mathbb{R}^{d_h \times d_{in}} \\ \mathbf{b}_1 &\in \mathbb{R}^{d_h} \\ \mathbf{W}_2 &\in \mathbb{R}^{1 \times d_h} \\ b_2 &\in \mathbb{R}\end{aligned}$$

Part b) (10 marks)

- **5 marks:** Conceptual argument that linear operations compose into linear operations.
- **5 marks:** Mathematical proof. If $f(x)$ is linear (e.g., $f(x) = x$), the network becomes:

$$\begin{aligned}\hat{y} &= \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + b_2 \\ \hat{y} &= (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + b_2)\end{aligned}$$

  This simplifies to $\hat{y} = \mathbf{W}_{eq}\mathbf{x} + b_{eq}$. Thus, without non-linearities, a deep network collapses mathematically into a standard single-layer linear model, losing all capacity to learn complex, non-linear representations.